ATANDA STUDIO · ARCHITECTURE REVIEW

ATANDA FE vs ARK-X

Comparative repository review and integration decision brief

Decision: retain ARK-X as the system of record. Do not perform a wholesale merge of atanda-fe. Selectively port or rebuild proven frontend capabilities behind ARK-X-owned contracts, authorization, and persistence.

EVIDENCE CUT
16 AUG → 06 SEP 2026

ATANDA

ATLAS COMPATIBLE

Executive verdict

ARK-X is the full-stack system of record. It contains the Express application, PostgreSQL/Drizzle schema and migration history, domain engines, server-side integrations, and committed tests. **atanda-fe is a sophisticated static frontend and service-facade candidate, not a replacement backend.** Do **not** wholesale merge it into ARK-X: its React Router composition, Axios service layer, client-side types, Cloudflare deployment shape, and admin routing make assumptions that do not match the ARK-X server contract or security boundary.

The sound course is selective reuse: preserve valuable presentation patterns and public/demo content where product ownership agrees, then rebuild each data-bearing feature against ARK-X APIs and server-enforced policy. A UI component is not proof that its associated workflow, authorization, storage, or external integration exists and is compatible.

> ***Non-claim:*** *this is a repository review, not a merge report. No union build, migration, integration test, penetration test, or acceptance run was performed or implied.*

1. Provenance, evidence limits, and measured comparison

REPOSITORY	EVIDENCE COMMIT	COMMIT TIMESTAMP (UTC)	COMMIT MESSAGE / ROLE
`oluseye4233/atanda-fe`	`5f8106521879648e4079dd79e524c1c0eeec987c`	2026-08-16 16:30:28	“Merge pull request #21 … analyze cv button”
`oluseye4233/ARK-X`	`dc172c425b252ba09d66fe4c3ceba916fa56a151`	2026-09-06 16:51:57	“Add ARK-X PDD (ATLAS 360 PLAN) draft”

The review is bounded to the supplied GitHub trees, comparison manifest, downloaded `atanda-fe` source, and `/tmp/arkx-remote` material corresponding to the requested ARK-X cut. The trees declare `truncated: false`. Current workspace files were used only as architectural orientation and are **not evidence of what existed at `dc172c`**. Findings below label ARK-X facts as remote-commit facts; no later local work is attributed to that commit.

EXACT COMPARISON FROM `REPO-COMPARISON.JSON`	COUNT	INTERPRETATION
atanda-fe files	**300**	Frontend-heavy tree
ARK-X files	**397**	Full-stack tree with migrations, server, tests and materials
Identical paths/content	**99**	Strong shared ancestry / copied foundation
Changed same-path files	**56**	Shared files have divergent revisions
atanda-fe-only	**145**	Mostly frontend organization, service/types, static resume and Cloudflare files
ARK-X-only	**242**	Includes **88 server** and **28 migration** files, tests, build/deploy support and assets

Counts are file counts, not a direct measure of product scope or quality. Assets and documents materially inflate them: ARK-X-only includes 35 assets and 69 “other” files (including PDD/export material), while atanda-fe-only includes 2 assets and 9 “other” files. Conversely, ARK-X’s server and migration counts are architectural evidence that static-file totals conceal.

Evidence references: `/tmp/github-compare/atanda-fe-tree.json`, `/tmp/github-compare/ARK-X-tree.json`, and `/tmp/github-compare/repo-comparison.json`.

2. Architecture comparison

CONCERN	ATANDA-FE @ `5F810…`	ARK-X @ `DC172…`	INTEGRATION CONSEQUENCE
Client routing	React Router v6, `BrowserRouter`, lazy public/auth/demo/admin partitions in `client/src/App.tsx`	Wouter `Switch` / `Route` in `client/src/App.tsx`	Route components cannot be dropped in unchanged. Choose one router per deployed client.
Runtime topology	Vite static client plus Cloudflare Wrangler deployment (`package.json` , `wrangler.jsonc`)	Node/Express server, Vite integration/static serving (`server/index.ts` , `server/vite.ts` , `server/static.ts`)	Static hosting must be deliberately connected to API/session origin.
Package/lock model	Bun lockfile (`bun.lock`); Vite/Wrangler scripts	npm lockfile (`package-lock.json`); Node/tsx build/start/migration/test scripts	Do not combine lockfiles or run scripts across roots.
Data/API facade	Axios-style `client/src/services/*` and locally owned `client/src/types/*` ; source inventory shows service imports	`client/src/lib/api.ts` fetch facade with `credentials: "include"` and Express routes	Recreate clients from server-owned DTOs; do not trust client types as the contract.
Auth/session	`AuthContext` , `ProtectedRoute` , and client `AdminGuard` are present in the atanda tree	Remote imports `requireAuth` , `requireSelf` , `requireInstructor` , `requireInstitutionAdmin` , and `loginSession` in `server/routes.ts` ; package includes Passport/session/PG session dependencies	Client guards are navigation UX only. ARK-X server policy must govern every protected action.
Persistence	No server or migration directory in tree	`shared/schema.ts` , Drizzle configuration, 28 migrations, `server/storage.ts`	ARK-X owns data evolution and transaction/tenant policy.
Tests	No test directory reported in the atanda tree	20 server test files reported, including scoring, cumulative merge, HR, matchmaking and email tests	Reused UI needs new component/e2e contract tests; test

			presence is not a passed-run claim.
AI/e mail	UI/service intents are visible in atanda inventory	`server/ai/*` , `server/mail.ts` , assessment/confirmation/nudge email modules and tests are remote facts	Preserve provider credentials and usage controls on server; never move them into browser code.

Build and deployment detail

atanda-fe scripts state `vite` , `vite build` , `tsc` , and `vite build && wrangler deploy` ; its dependencies include `react-router-dom` , Axios and Wrangler. ARK-X scripts state `tsx server/index.ts` , bundled production start, `drizzle-kit` , migrations, test commands, and PDD rendering; its package declares Express, `pg` , Drizzle, Passport, session store, Helmet, Multer, rate limiting, AI SDKs and mail/PDF tooling. These are incompatible operational models unless designed as separately deployed applications.

3. Shared ancestry and divergent UI inventory

The 99 identical files demonstrate a shared UI base: many Radix-style primitives under `client/src/components/ui`, dashboard visualizations (`ArkIdentityCard`, `JSTRadar`, `TaskHeatmap`, `Vulnerability*`), report/export helpers, baseline hooks, selected pages, and shared domain utilities such as `shared/assessmentMerge.ts` and `shared/claimMatch.ts`. The 56 changed shared paths include both routers, core styles, auth hooks, dashboard/report/marketplace/profile pages, `shared/schema.ts`, feature flags, and build configuration. Shared ancestry is therefore useful for visual extraction, **not** evidence that either side remains behaviorally interchangeable.

DIVERGENT INVENTORY	ATANDA-FE-SPECIFIC EVIDENCE	ARK-X-SPECIFIC EVIDENCE AT REMOTE CUT	REVIEW READING
Public/auth/demo shell	Home section components; forgotten/reset/verify/signup pages; `DemoApp` and six demo subpages	Larger remote home/login/demo pages, but no atanda app partition	atanda has a deliberately partitioned marketing/demo/auth composition.
Admin command centre	`app/AdminApp.tsx`; admin dashboard, users, plans, payments, AI, F1000 and flags pages	`server/routes.ts` includes admin endpoint logic; tree has only `pages/admin-ccge-import.tsx`	Admin UI breadth is atanda value, but it must be rebuilt with server scopes.
Profile/account	Seven profile tabs: profile, privacy, security, subscription, portfolio, marketplace, AI	Remote profile page is larger and server-backed API exists	Reconcile UX and data model; no direct type copying.
Marketplace	Listing, publish, filters, price/performance/AI analysis components	marketplace, Bonsai and Forge Lab pages; server Sphinx/Forge modules	ARK-X is better placed for state-changing marketplace workflows.
Resume	Static public “living-resume” HTML/PDF and ARK resume components	Living Resume Designer page/components/export plus `server/livingResumeDesigner.ts`	Static public artifact stays separate; designer is a server-backed ARK-X feature.
Domain engines	Service/type layer only in atanda inventory	scoring, recomputation, matchmaking,	Keep computational authority in ARK-X.

		workforce/connectors, AI, mail, storage	

4. Detailed atanda-fe capability review

Structure and delivery

`client/src/App.tsx` makes public pages eager and lazy-loads `AuthenticatedApp`, `DemoApp`, and `AdminApp`. The latter is selected when `window.location.hostname.split(".")[0] === "atanda-admin"`; `AdminApp` mounts `AdminGuard` and routes dashboard, users, plans, payments, AI, F1000 and feature flags. This is a clean client bundle organization and a useful command-centre UX inventory. It is **not authorization**: hostname selection and a React guard can be bypassed by direct API calls unless every server endpoint checks an authenticated principal and scope.

`AuthenticatedApp.tsx` explicitly labels currently mounted routes as career assessment, games, marketplace, book, F1000, GUIN public profile and account chrome. Its source comment says marketplace, matchmaking, workforce and training were intentionally excluded until revived, yet those page files exist in the tree. Thus:

ATANDA ROUTE STATUS AT THE STATED COMMIT	EVIDENCE	TREATMENT
Routed public/auth	`/`, login, signup, account/reset verification, privacy/terms, payment outcome, `/demo-tour/*` in `App.tsx`	Candidate for content/design extraction after legal and identity review
Routed authenticated	`/upload`, `/assessment`, `/dashboard`, `/pathways`, `/ark-resume`, `/report`, `/ark/history`, `/play`, `/marketplace/*`, `/book`, `/f1000`, `/u/:username`, profile/subscription/checkout in `AuthenticatedApp.tsx`	Rebuild against ARK-X contracts
Routed admin (subdomain branch)	Admin dashboard/users/plans/payments/AI/F1000/flags in `AdminApp.tsx`	High-value UX, redesign authorization first
Present but not mounted by	`matchmaking*`, `workforce`, `training*`, `school-dashboard`, `enterprise`, `shared-report`, `confirm-invite`, `context-craft`, demo subpages	Treat as dormant/unrouted

`AuthenticatedApp`		inventory, not shipped capability

Capability observations

- **Public/auth/demo:** reusable presentation patterns exist in `components/home/*`, `components/auth/*`, demo layouts and pages. Password/OTP/reset UI must map to real ARK-X lifecycle endpoints before reuse.
- **Admin command centre:** `components/admin/*` offers layout, status badges, pagination, confirmation/error patterns and pages. The pages depend on services/types (`users`, `plans`, `payments`, `feature-flags`, `ai`) that are atanda-owned assumptions; do not make them authoritative.
- **Services/types:** the tree contains 23 `services/*.service.ts` and 22 `types/*.ts` entries. They provide a useful consumer-inventory and error handling direction (`lib/apiError.ts`), but duplicate DTO ownership risks drift.
- **Profile/privacy:** `components/profile/{Profile,Privacy,Security,Subscription,Portfolio,Marketplace,AI}Tab.tsx` are valuable UX candidates. Privacy/security controls must only claim settings ARK-X actually persists/enforces.
- **Marketplace/report/dashboard:** marketplace components, report/dashboard charts, pathways and ARK resume display provide mature visual vocabulary. Most must be adapted because score/status/pagination and ownership semantics are server-defined in ARK-X.
- **Living resume static page:** `client/public/living-resume-oluseye-shay-amusa/index.html` and its PDF are static public assets, not the remote ARK-X designer. Keep only with explicit content-owner approval; do not treat it as authenticated resume data.
- **Design/refactoring guidance:** `.kiro/steering/design-system.md`, `instructions.md`, and `refactoring-standards.md`, plus `LL_UI_INTEGERATION.md`, are useful process/design input but not production contracts.

5. Detailed ARK-X review — remote `dc172c` facts only

At the 06 Sep remote cut, ARK-X contains a full application boundary: `server/index.ts`, `server/routes.ts`, `server/storage.ts`, `shared/schema.ts`, Drizzle configuration and migrations `0000` – `0024`. `shared/schema.ts` centralizes subscription plans, score glossary, JST/CCMI weights, tier/status constants, validation schemas and persistence definitions. For example, it defines ARK as JST + CCMI (0–600), JST weights, CCMI pillar weights, and certification thresholds. This supports ARK-X as the contract and calculation authority.

REMOTE-COMMIT CAPABILITY	EVIDENCE PATHS/SYMBOLS	MATURITY INFERENCE
Auth and protected routing	`server/auth.ts`; `requireAuth`, `requireSelf`, `requireInstructor`, `requireInstitutionAdmin` imported by `server/routes.ts`	Server policy primitives exist; endpoint-by-endpoint audit still required
Database/domain	`shared/schema.ts`, `server/storage.ts`, `migrations/0000…0024`	Persisted model and evolution mechanism are present
Scoring / ARK recalculation	`server/scoringEngine.ts`, `server/arkRecalc.ts`, `server/arkBackfill.ts`; `server/__tests__/scoring.test.ts`, `arkRecalcCaps.test.ts`	Engine and targeted test source are present
Cumulative assessment/resume	`server/resumeAnalyzer.ts`, `profileExtract.ts`, `shared/assessmentMerge.ts`, migrations 0009–0010, merge test	Server pipeline is present
Marketplace and synthesis	`server/sphinx.ts`, `sphinxSynergy.ts`, `synthesis.ts`, `forgeLab.ts`, migrations 0004–0007	State and domain modules are present
Workforce	`server/hrConnectors/*`, `hrScheduler.ts`, `workforceConnectionTest.ts`, migrations 0011–0012 and HR tests	Connector/sync architecture is present
Matchmaking/training	`server/matchmaking.ts`, `trainingMatch.ts`, migrations 0020/0017, tests	Domain-backed capability is present

Living Resume Designer	`pages/living-resume-designer.tsx`, `components/living-resume-designer/*`, `server/livingResumeDesigner.ts`, `client/src/lib/livingResumeExport.ts`	Dedicated designer/prefill/export path exists
Gauntlet	`server/gauntlet.ts`, `pages/gauntlet.tsx`, migration 0024	Persisted run feature exists
AI and email	`server/ai/{client,providers,usage,…}.ts`, `mail.ts`, confirmation/assessment/nudge mail modules and tests	Server-side integration layer exists
Security controls	`helmet`, rate-limit dependency; rate limiters and file limits/mime filters in `server/routes.ts`; `threat_model.md` in tree	Concrete controls exist, but this review does not certify them

The remote `client/src/lib/api.ts` uses same-origin relative `/api` requests and `credentials: "include"`. It exposes endpoints across login, assessment, enterprise/workforce, CCGE/verification, marketplace, reports, training, ARK resume, Living Resume telemetry and Gauntlet. That is evidence of a broad client-server contract. It is not proof that every endpoint was deployed, accessible, secure, or tested successfully.

6. Capability matrix

CAPABILITY	ATANDA-FE	ARK-X REMOTE	MATURITY ASSESSMENT	REUSE RECOMMENDATION
Marketing/public shell	Yes	Yes	Divergent implementations	Keep atanda marketing/demo separately or extract design tokens
Authentication UX	Yes	Yes	Contract differs	Redesign against ARK-X session endpoints
Server auth/RBAC	No server evidence	Yes	ARK-X foundation	Keep ARK-X; strengthen scopes
Admin command centre UI	Yes	Partial page/UI evidence	UI ahead of remote admin surface	Redesign before reuse
Assessment/dashboard/report	Yes	Yes	Shared ancestry, changed pages	Port visual slices after DTO mapping
Score calculation	UI/display only	Engine/schema/tests present	ARK-X authoritative	Do not port atanda logic
Marketplace	Rich UI	Full domain modules/migrations	UI + engine diverge	Vertical-slice rebuild
Workforce	Page exists, unrouted	connectors/sync/tests/migrations	ARK-X substantive	Keep ARK-X backend; rebuild UI
Matchmaking	Pages/services present but unrouted	engine/test/migration	ARK-X substantive	Reuse visual patterns only
Living resume	static page/ARK resume UI	designer, server prefill/export	Separate products/artifacts	Keep static asset separate; integrate designer
AI/email	service intents	provider/mail modules/tests	Server-sensitive	ARK-X only
Tests	Not evidenced	20 test files	Targeted source exists	Extend ARK-X suite; do not claim passing

7. API and data-contract compatibility

CONTRACT AREA	ATANDA-FE ASSUMPTION/EVIDENCE	ARK-X REMOTE FACT	REQUIRED RECONCILIATION
Transport	Axios-like `services/*` and `BASE_URL` export indicated by `services/index.ts`	`fetch`, relative `/api`, `credentials: "include"` in `client/src/lib/api.ts`	Adopt one generated/handwritten ARK-X client; configure origin intentionally
DTO ownership	Local `client/src/types/*` by domain	Shared schema/constants and server validation in `shared/schema.ts` /routes	Server/shared contract owns DTOs; map view models at UI edge
Authentication	Context/route guard	Session-derived identity; `requireAuth` family	No user ID accepted as authority; derive principal server-side
Authorization	Admin subdomain + `AdminGuard`	server role/institution middleware symbols	Define scope matrix: platform admin, institution admin, instructor, self
Pagination/search	atanda has `PaginationControls`; service-specific unknowns	ARK-X workforce API uses `q`, `limit`, `offset` in `api.ts`	Specify bounds, defaults, total/count envelope and invalid-value behavior
Errors/status	`lib/apiError.ts` and service error assumptions	`apiRequest` throws server `message`; routes use HTTP statuses	Publish an error envelope/code taxonomy; UI must handle 401/403/404/409/422/429
Enums	Client-owned plan/payment/verification types	`SUBSCRIPTION_PLANS`, `SPC_STATUSES`, `OPPORTUNITY_STATUSES`, confirmation enums in schema	Import/shared-generate enums; version changes deliberately
Admin operations	users/plans/payments/AI/flags UI	Remote endpoint inventory and env-admin checks exist	Introduce explicit privileged DTOs, audit actor/action and least privilege

8. Security and privacy review

1. **Client guards are not authorization.** `ProtectedRoute`, `AdminGuard`, lazy chunks and hidden navigation improve UX only. Enforce authentication, ownership and scope on ARK-X handlers.
2. **Hostname-selected administration is insufficient.** The `atanda-admin` prefix branch in atanda `App.tsx` must not be a trust decision. Bind admin privileges to server-side roles/scopes and test denial paths.
3. **Tenant and institution boundaries:** workforce, cohorts, staff, reports and marketplace operations need server-side institution/tenant predicates, object ownership checks, and audited overrides; never accept a client user/institution ID as authority.
4. **Sessions and split origins:** ARK-X's API client uses cookies. If Cloudflare frontend and Node API differ by origin, set exact CORS allowlists, `credentials`, cookie `Secure` / `HttpOnly` / `SameSite`, CSRF protection for cookie-authenticated mutations, and no wildcard credentialed CORS. Same-origin avoids much of this complexity.
5. **PII and uploads:** minimize fields sent to browser/telemetry; define retention/deletion and export access rules. Validate upload content server-side, not MIME alone; ARK-X's size/type limits are a start, not a certification.
6. **AI/email/payment:** keep keys, provider calls, rate/budget policy and recipient decisions server-side. Ensure consent, template review, idempotency and audit events. Do not expose source text, resumes or contact data in client logs.
7. **Operational controls:** evaluate feature flags server-side for privileged features; rate-limit abuse-prone routes; use structured security telemetry without raw PII; add security headers and dependency/secret scanning in CI.

9. Deployment comparison and recommended topology

MODEL	BENEFITS	CONSTRAINTS
atanda static Cloudflare Pages/Workers	Fast static delivery; independently deployable marketing/demo	Cross-origin cookies/CSRF/CORS and duplicated auth/API contract become risks
ARK-X Replit Node/Express autoscale	Same-origin API/session potential; database and worker-facing runtime co-located	Application and public content release cadence are coupled
Reverse proxy / same-origin integration	Static edge can front `/`, proxy `/api` to ARK-X	Requires explicit cookie, cache, header, health and rollback design

Recommended topology: make ARK-X Node/Express + PostgreSQL the authoritative application/API origin. Keep atanda marketing/demo/static living-resume material as a separately deployed public property only when it needs independent cadence. Route product authentication and authenticated UI through a same-origin ARK-X shell (or a rigorously tested reverse proxy that preserves `/api` same-origin semantics). Do not point a static client at a permissive cross-origin credentialed API as the default integration.

10. Classification

CLASSIFICATION	ITEMS	DECISION
Port directly (after licensing/accessibility review)	Identical UI primitives; small visual atoms; generic layout/status/pagination patterns	Copy only when dependencies and accessibility behavior remain compatible
Redesign before reuse	Auth, admin command centre, profile/privacy/security, dashboards/reports, marketplace and payment surfaces	Preserve UX intent; bind to ARK-X DTOs, policies and errors
Keep separate	Marketing/demo shell, Cloudflare deployment assets, static living resume/public collateral	Link or brand-align; do not couple to authenticated data
Duplicate / do not port	atanda `services/*`, client `types/*` as authorities, React Router app partitions, hostname admin selection, client-side access rules	ARK-X server/shared contracts replace them

11. Phased roadmap and effort

Broad estimate: 30–50 engineer-days. Assumptions: one experienced full-stack engineer plus timely product/security review; ARK-X remote baseline is buildable; no major database rewrite; selected UI rather than every atanda screen; existing CI/environment access. It excludes new payment-provider production certification, full design-system rewrite, data migration from an unknown external store, and a formal penetration test.

PHASE	SCOPE AND EXIT	ROUGH EFFORT
0. Freeze and contract inventory	Tag both references; route/page/service inventory; choose canonical DTO/error/status envelopes; identify content ownership	3–5 days
1. Policy and contracts	Server RBAC/scope matrix, tenant predicates, admin audit events, CSRF/CORS/session decision, DTO adapters	6–9 days
2. Low-risk UI extraction	Bring approved primitives, public presentation and shared visual tokens into ARK-X without data behavior	4–6 days
3. Vertical slices	Rebuild selected dashboard/report/profile/admin/marketplace slices one capability at a time with API contract tests	12–20 days
4. Hardening/release	Staging migration rehearsal; security/accessibility/load checks; observability, feature flags and rollback rehearsal	5–10 days

12. Exact update list

atanda-fe updates (only if retained as a client/property)

1. Replace the service/type authority under `client/src/services/*` and `client/src/types/*` with ARK-X-owned, versioned DTO adapters; remove assumptions that client IDs confer authority.
2. Replace the `atanda-admin` hostname decision in `client/src/App.tsx` and client `AdminGuard` trust with server-issued capabilities; retain only UI gating.
3. Map `AuthContext`, login/reset/verification views and `ProtectedRoute` to ARK-X session semantics; implement 401/403/CSRF/session-expiry states.
4. Remove or explicitly label unmounted page files as dormant; do not market them as shipped routes. Reconnect only through accepted vertical slices.
5. If it remains split-origin, set deliberate API base URL, credentialed CORS/CSRF/cookie policy and no secret-bearing client configuration.
6. Remove or replace static named living-resume material unless its publishing consent, ownership and privacy review are documented.

ARK-X updates

1. Make `shared/schema.ts` /server validators the published DTO, enum and status source; add response envelopes and contract/version tests for adopted screens.
2. Audit every privileged `server/routes.ts` handler for `requireAuth` plus self/institution/admin scope, including list/search/export and indirect object references.
3. Add explicit platform-admin versus institution-admin scopes; do not rely solely on `ADMIN_USER_ID` environment comparison for a growing command centre.
4. Add audit logging for admin, data export, billing/plan, staff and feature-flag actions; redact PII from telemetry.
5. Add UI/component/e2e coverage for ported flows in addition to existing remote server tests, and run migration compatibility in staging.
6. Establish a route/feature-flag ownership map so the Wouter app exposes only approved surfaces.

Shared reconciliation tasks

- Select a router/runtime owner, a canonical API client and error taxonomy.
- Produce mapping sheets for user/profile, assessment/ARK, subscription, listing, workforce, confirmation and admin DTOs.
- Agree public URL, brand, analytics, consent, retention and deletion policy.
- Decide which source assets/documentation transfer; scan rights, sensitive content and dependency licenses.

13. Validation, acceptance, and rollback

ACCEPTANCE AREA	EVIDENCE REQUIRED BEFORE RELEASE	OWNER
API contracts	Versioned request/response fixtures; invalid enum/pagination/error cases	Backend + frontend
Authorization	Automated 401/403 and cross-tenant/cross-user tests for every adopted admin/workforce/export action	Backend/security
Data	Migration rehearsal, backup/restore proof, no unintended schema drift, PII redaction review	Data/backend
UX/accessibility	Keyboard, focus, form errors, responsive checks and WCAG-oriented review of ported views	Frontend/design
Security	Session/CSRF/CORS/cookie verification; upload/telemetry/secret review; rate-limit checks	Security/backend
Performance	Representative dashboard/list/upload and API load checks; cache headers for public assets	Platform
Release	Feature-flagged staged rollout, monitoring/alerts, support playbook	Product/platform

Rollback approach: use additive, backward-compatible migrations first; take a verified backup before destructive changes; deploy server DTO support before enabling new UI; retain the prior client artifact and feature flag; disable the new vertical slice on elevated errors or authorization anomalies; restore application version before considering a data restore. Avoid irreversible data transformations until acceptance evidence is signed off.

14. Ranked risk register

RANK	RISK	WHY IT MATTERS	MITIGATION
1	Authorization bypass	Client/admin hostname logic could expose privileged operations	Server scopes, tenant predicates, denial tests and audits
2	Contract/data drift	Axios/types and ARK-X fetch/schema represent separate assumptions	Canonical DTOs, adapters, contract tests
3	Split-origin session failure	Cookies, CSRF and CORS break or weaken security	Prefer same origin; otherwise explicit hardened policy
4	Scope illusion	Unrouted atanda pages can be mistaken for deployed capability	Route inventory and feature acceptance per vertical slice
5	PII leakage	Resumes, staff records, confirmations and telemetry are sensitive	Minimization, consent, redaction, access logs and exports policy
6	Migration/release mismatch	UI ships ahead of schema/server availability	Additive releases, staging rehearsal, feature flags
7	AI/email cost or disclosure	Client-porting could bypass server usage/recipient controls	Keep providers/mail server-side with quotas and audit
8	Design regression	Direct port across router/state systems can introduce brittle UI	Extract atoms; redesign data-bearing flows

Immediate next action

Run a **contract-and-authorization workshop** before any code transfer: select the first vertical slice (recommend profile/ARK dashboard, not admin or workforce), publish its ARK-X request/response/error/scope matrix, and reject any atanda component whose inputs cannot be mapped without client-side authority.

Decision recommendation

Approve selective frontend reuse; reject a wholesale merge. Keep ARK-X at `dc172c` as the backend, database and domain system of record. Preserve atanda-fe’s strongest value—public/demo composition and command-centre/dashboard UX—as a curated design source, then rebuild approved slices behind ARK-X-owned contracts, RBAC, tenant controls and staged validation.